7.5 Let the failure reach someone who can handle it

Keep the model and custom exceptions from 7.4. Now we connect exception handling to the controller structure from class 6. We will catch a negative value in the page controller but deliberately leave the excessive case unmatched to show propagation. In 7.6, both known input failures will receive correction responses.

Keep response preparation separate from sending

Create `src/Tutorials/Class7/Response.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class Response
{
    public function __construct(
        public readonly string $body,
        public readonly int $status = 200,
    ) {
    }
}
```

Response is a small holder for the complete HTML and its status. The readonly properties are readable but cannot be reassigned after initialization. We use this object so a controller returns both values without sending partial output or headers. It is a course helper, not a framework requirement.

Create `src/Tutorials/Class7/Controller.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use Twig\Environment;
use Twig\Loader\FilesystemLoader;

abstract class Controller
{
    protected Environment $twig;

    public function __construct()
    {
        $loader = new FilesystemLoader(__DIR__ .
'/../../../templates/tutorials/class7');
```

```
        $this->twig = new Environment($loader, [
            'autoescape' => 'html',
            'strict_variables' => true,
        ]);
    }
}
```

Create `src/Tutorials/Class7/DessertController.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class DessertController extends Controller
{
    public function preview(int $calories): Response
    {
        $dessert = null;
        $error = null;
        $status = 200;

        try {
            $dessert = new Dessert('Berry crumble', $calories);
        } catch (NegativeCaloriesException $e) {
            $error = 'Use zero or a positive calorie count.';
            $status = 422;
        }

        $body = $this->twig->render('preview.html.twig', [
            'dessert' => $dessert,
            'error' => $error,
        ]);
        return new Response($body, $status);
    }
}
```

On failure, dessert stays null and the success branch cannot use an unconstructed result. Rendering occurs after this narrow domain catch. A Twig failure therefore propagates to the boundary rather than being mislabeled as bad calorie input.

Create `src/Tutorials/Class7/Application.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class Application
```

```
{
    public function run(): Response
    {
        $action = $_GET['action'] ?? 'valid';
        if (!is_string($action)) {
            $action = 'valid';
        }

        $controller = new DessertController();
        switch ($action) {
            case 'negative':
                return $controller->preview(-1);
            case 'excessive':
                return $controller->preview(5001);
            case 'valid':
            default:
                return $controller->preview(500);
        }
    }
}
```

Application selects a route and returns the controller's response. It contains no redundant catch: it has no additional recovery action to perform. Each route supplies one known test value. Missing, unknown, and array-shaped action values select the valid demo at this stage.

Create the templates

Create `templates/tutorials/class7/` outside public/.

Create `templates/tutorials/class7/base.html.twig`:

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Campus Dessert Planner: {% block pageTitle %}Planner{% endblock %}
</title>
    <style>
        body { font-family: system-ui, sans-serif; max-width: 48rem; margin: 2rem
auto; padding: 0 1rem; }
        nav a { display: inline-block; margin-right: 1rem; }
        label { display: block; margin-top: 1rem; }
        input, button { font: inherit; padding: .4rem; }
        button { display: block; margin-top: 1rem; }
        .error { border-left: .3rem solid #a02020; padding-left: 1rem; }
        .notice { background: #edf4ff; padding: 1rem; }
    </style>
</head>
<body>
    <nav aria-label="Demonstrations">
```

```
        <a href="index.php">Planner</a>
        <a href="index.php?action=valid">Valid demo</a>
        <a href="index.php?action=negative">Negative demo</a>
        <a href="index.php?action=excessive">Excessive demo</a>
    </nav>
    <h1>{{ block('pageTitle') }}</h1>
    {% block main %}{% endblock %}
</body>
</html>
```

Create `templates/tutorials/class7/preview.html.twig`:

```
{% extends 'base.html.twig' %}

{% block pageTitle %}Dessert preview{% endblock %}

{% block main %}
    {% if error is not null %}
        <p class="error" role="alert">{{ error }}</p>
    {% elseif dessert is not null %}
        <p>Preview ready: {{ dessert.name }}, {{ dessert.calories }} calories.</p>
    {% endif %}
{% endblock %}
```

Twig reads name and calories through the public model getters. Explicit null context values matter with strict_variables enabled: a missing key differs from a key whose value is null.

Add the last-resort entry boundary

Replace `public/tutorials/class7/index.php`:

```
<?php
declare(strict_types=1);

use App\Tutorials\Class7\Application;

try {
    require_once __DIR__ . '/../../../vendor/autoload.php';
    $app = new Application();
    $response = $app->run();
} catch (Throwable $e) {
    // Next week: record private diagnostic evidence here with a logger.
    http_response_code(500);
    header('Content-Type: text/html; charset=utf-8');
    echo '<!doctype html><html lang="en"><head><meta charset="utf-8">';
    echo '<title>Request failed</title></head><body><h1>Request failed</h1>';
    echo '<p>We could not complete this request. Please return to the planner.
</p>';
    echo '<p><a href="index.php">Return to the planner</a></p></body></html>';
```

```
    return;
}

http_response_code($response->status);
header('Content-Type: text/html; charset=utf-8');
echo $response->body;
```

The try covers autoloading, application construction, execution, and rendering. The fallback uses fixed HTML directly, so it does not depend on the failing controller or Twig setup. No model or template has echoed anything before the complete Response is available. The entry sends one response, with headers before body output.

The catch uses Throwable because an unexpected TypeError or other Error object also needs a controlled failure response. This is a last-resort boundary, not a claim of successful recovery. The top-level return ends this entry script's work; there is no success output afterward. Do not call run again or use `$response` in the failure branch.

The commented logger integration point is for next week. This example does **not** record the caught diagnostic evidence, so a generic page alone is not enough for diagnosing production failures. During local development, deliberately reproduce and inspect defects, then fix them. Configure public error display off in a public deployment; catching Throwable does not suppress ordinary warnings or errors that prevent the entry script from parsing.

Trace and verify in the browser

Start the server from the student repository root and visit the nested URL:

```
php -S localhost:8000 -t public
```

URL suffix after /tutorials/class7/	Expected body	HTTP status
index.php	Berry crumble, 500 calories	200
index.php?action=valid	Same valid preview	200
index.php?action=negative	Correction, no preview ready	422
index.php?action=excessive	Generic Request failed page	500
index.php?action=unknown	Valid preview	200
index.php?action[]=negative	Valid preview	200

Use the browser Network panel to inspect status; body text alone cannot prove it. All navigation is relative to this week's index.php.

Trace the excessive case: setter throws → constructor unwinds → preview has no matching catch → run unwinds → index catches Throwable. A negative value stops earlier in preview. The entry boundary is a good place to end an unhandled request; it is a poor place to decide which form field to correct because the controller has that context.